

Reinforcement Learning: When the Signal is a Scalar Reward Over Trajectories

The closing post of the inductive-bias series. The previous six posts varied two parallel axes: the architecture (MLP, CNN, RNN, Bengio fixed-context, Transformer) and the output head (sigmoid, softmax, Poisson, heteroscedastic Gaussian, mixture). Both axes lived inside **supervised learning**. The training signal was always a label per example, and the gradient question was always “how do I move $f_\theta(x)$ closer to this specific y .”

Reinforcement learning is the paradigm where the signal disappears at the example level and reappears as a scalar reward summed along a trajectory. The distinctive hard part is not the absence of labels. It is **temporal credit assignment**: a trajectory of two hundred actions produces one number, and the learner has to figure out which of those actions mattered. Every named technique in RL (Monte Carlo, TD, GAE, eligibility traces, hindsight relabeling, off-policy correction) is a piece of credit-assignment machinery. Supervised learning never needs that machinery because it never has the problem.

This post does the minimum to draw the line between supervised and RL precisely. It derives REINFORCE, reads the result back as “supervised cross-entropy weighted by return,” and trains a tiny policy on a 5×5 gridworld. Then it points at AIXI as the theoretical north star of the dedicated RL series, the way Solomonoff induction (in the Bengio LM post) framed the language-modeling arc.

1. The trichotomy

Three standard paradigms, with the boundary that matters drawn at the right place.

- **Supervised.** Given (x, y) pairs, learn f_θ such that $f_\theta(x) \approx y$. One gradient per example. The loss tells you the answer was wrong, *and how to be less wrong on this exact input*.
- **Unsupervised.** Given x alone, learn structure: density, clusters, manifolds, factors. No target.
- **Reinforcement learning.** Given an environment and a scalar reward, learn a policy $\pi_\theta(a | s)$ that maximizes expected return. The signal is one number per trajectory, not a label per example.

Where SSL fits. Self-supervised learning (next-token, masked patch, contrastive partner) is supervised learning with auto-generated labels. Mathematically identical. Practically distinct because the data supply is unbounded, which is what changes the scaling story (a Transformer trained on 10TB of web text is doing the same loss minimization as a 1-layer softmax classifier on UCI digits, just with more data and a heavier model). SSL is not a fourth paradigm; it is supervised learning that solved its label problem.

The boundary that matters is not “do we have labels” but “do we have *per-example* gradient signal.” Supervised and SSL do. RL does not.

2. The supervised-to-RL reduction loses what makes RL hard

A common move: “supervised learning is just RL with reward $r = -\text{NLL}(y | x)$ at every step.” Technically true. The reduction packages each (x, y) pair as a one-step episode in which the agent takes action $\hat{y}$, the environment immediately reveals y , and the reward is the negative log-likelihood of y under $\pi_\theta(\cdot | x)$. Maximizing expected reward is maximum likelihood.

What that reduction loses:

- **Delayed reward.** Reward arrives at the same step as the action, not many steps later.
- **Sparse reward.** Every action has a reward, not “one reward per episode at the end.”
- **Exploration.** The “action” $\hat{y}$ and the gradient of reward with respect to it are directly available; no need to try the alternatives.
- **Off-policy correction.** The data and the policy are the same thing; there is no distinction between behavior and target.

Each of those losses is a hard part of RL erased. The genuine difficulty is temporal credit assignment. A trajectory $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T)$ produces a single return $R(\tau) = \sum_t r_t$. The learner took, say, 200 actions. Which ones were responsible for the return being high or low?

REINFORCE, advantage estimation, TD learning, GAE, eligibility traces, hindsight relabeling, off-policy corrections: every named technique is a piece of that credit-assignment machinery. Supervised learning never needs it because the label tells you precisely which input produced which loss, and the chain rule walks the blame backward through the network. RL’s loss surface, by contrast, is computed *after* the actions are committed, and the trajectory probability is a product over many policy decisions of which only some mattered.

The reframing matters because it lets a reader who has only done supervised work see *why* RL is hard, not merely *that* it is.

3. The MDP framing

A **Markov decision process** is the standard formalism. An agent and an environment interact in a loop. At step t the agent observes a state s_t , takes an action $a_t \sim \pi_\theta(\cdot \mid s_t)$, and the environment returns a reward r_{t+1} and a next state $s_{t+1} \sim p(\cdot \mid s_t, a_t)$.

Markov property. The transition distribution $p(s', r \mid s, a)$ depends only on the current (s, a) , not on the prior history. The “state” is whatever you need to remember.

Discount. Choose $\gamma \in [0, 1)$. The discounted return from step t is

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

γ is geometric weighting on the future. $\gamma = 0$ is myopia: only the next reward matters. $\gamma \rightarrow 1$ is the undiscounted case: a step now and a step in a thousand are valued equally. In practice $\gamma \in [0.9, 0.999]$ is typical; the effective horizon is $1/(1 - \gamma)$ steps.

Objective. Maximize the expected return

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)],$$

where $\tau = (s_0, a_0, r_1, s_1, \dots)$ is a trajectory sampled by following π_θ in the environment and $R(\tau) = \sum_{t \geq 0} \gamma^t r_{t+1}$.

Value functions. The state-value and action-value functions are

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

A policy is a conditional distribution over actions given states; a return is the (discounted) sum of rewards along the trajectory the policy induces; we want the policy whose induced trajectories have the highest expected return; V^π and Q^π are the natural quantities to estimate along the way.

4. Credit assignment, three families

Supervised: every example carries its own loss; the chain rule pins the blame precisely.

RL: one return per episode (or a few sparse rewards along the way), a sequence of many actions between them, and the question “which action mattered” is genuinely open. Three families of answers, each a different bet about how to split the credit:

- **Monte Carlo.** Wait until the episode ends. Attribute the entire realized return G_t to *every* action taken in the trajectory, optionally discounted by recency. High variance (one trajectory is one sample), unbiased (the return is what actually happened).
- **Temporal difference (TD).** Do not wait. Use a learned value estimate $V(s_{t+1})$ as a bootstrap target for $V(s_t)$ via the Bellman equation. Lower variance (single-step targets), biased (by the accuracy of V).
- **Policy gradient.** Skip the per-trajectory credit-assignment question. Estimate the gradient of $J(\theta)$ as an expectation over trajectories, average many samples, and let the averaging do the work of variance reduction.

Actor-critic, n -step returns, GAE, and eligibility traces are interpolations between these three: bias-variance trade-offs on the shape of the credit signal.

This post focuses on the policy-gradient family because the bridge to supervised cross-entropy is exact and the derivation is short.

5. REINFORCE, derived

Parameterize the policy: $\pi_\theta(a \mid s)$. Think “neural network outputting a softmax over discrete actions” or “neural network outputting the parameters of a continuous distribution.” The gridworld example below uses the discrete-softmax form.

The objective is the expected return under the policy-induced trajectory distribution:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \sum_{\tau} p_\theta(\tau) R(\tau),$$

with $p_\theta(\tau)$ the probability of sampling trajectory τ . We want $\nabla_\theta J(\theta)$.

The log-derivative trick. For any parameterized distribution p_θ ,

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \cdot \nabla_\theta \log p_\theta(\tau).$$

This is just $\nabla \log x = \nabla x / x$ rearranged, but it is the load-bearing identity of the policy-gradient family. It turns a gradient of a probability (hard, because probabilities are normalized) into a gradient of a log-probability (easy, because of the sum factorization below) times the probability itself (which becomes an expectation when we sample).

Apply it:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} p_{\theta}(\tau) \cdot R(\tau) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot R(\tau)].$$

Now factor the trajectory probability. A trajectory is sampled by alternating policy choices and environment transitions:

$$p_{\theta}(\tau) = p(s_0) \prod_{t \geq 0} \pi_{\theta}(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t).$$

Taking the log and the gradient with respect to θ :

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t).$$

The initial-state distribution and the transition kernel do not depend on θ , so their gradients vanish. Only the policy factors survive. This is the central structural fact that makes model-free RL possible: you can estimate the policy gradient without knowing the environment's dynamics.

Substituting, the **reward-to-go** form of REINFORCE drops out by a causality argument (a reward earned before step t cannot have been influenced by the action a_t , so its contribution to the gradient averages to zero):

$$\boxed{\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) \cdot G_t \right].}$$

In words: on average across sampled trajectories, the gradient of the expected return is the sum over time steps of the log-likelihood gradient of the action taken, weighted by the return earned from that step onward. Sample one trajectory, compute that sum, and you have an unbiased Monte Carlo estimate of $\nabla_{\theta} J$. Step in its direction with SGD.

Variance reduction by a baseline. Subtracting any state-dependent baseline $b(s_t)$ from G_t leaves the expectation unchanged but typically reduces variance:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) \cdot (G_t - b(s_t)) \right].$$

The expectation is unchanged because $\mathbb{E}_a [\nabla_{\theta} \log \pi_{\theta}(a \mid s)] = 0$ for any s , so multiplying by a state-dependent constant and summing gives zero. The variance change depends on how strongly $b(s_t)$ correlates with G_t ; a learned value estimate $V_{\phi}(s_t)$ is the standard choice, and $G_t - V_{\phi}(s_t)$ is an estimate of the advantage $A^{\pi}(s, a)$. That is the half-step from REINFORCE to actor-critic.

The gridworld example below uses a simpler scalar baseline: a moving average of recent episode returns. It is enough to make the gradient useful at this scale.

6. The bridge: REINFORCE is weighted cross-entropy

Look at the per-step term in the REINFORCE update:

$$-\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$$

The first factor is *exactly* the supervised cross-entropy gradient on the pair (s_t, a_t) , treating a_t as a one-hot label and $\pi_{\theta}(\cdot | s_t)$ as the predicted class distribution. It is the same expression `scratchnn` derives in `SoftmaxCrossEntropy`: $\partial L / \partial z = p - \text{onehot}(a)$.

Policy gradient is therefore supervised cross-entropy on the actions the agent took, with each per-sample loss term reweighted by the return G_t (or by an advantage).

- **Behavioral cloning** is the special case where the weight is constant: given a dataset of expert (s, a) pairs, fit π_{θ} by softmax cross-entropy. Pure supervised learning of a policy.
- **REINFORCE** is the same softmax cross-entropy on the actions the *agent* took, weighted by how good the return turned out to be.
- **Actor-critic** is the same softmax cross-entropy weighted by an advantage, with a learned baseline subtracted from G_t .

In supervised learning, the dataset tells you what the right action was. In RL the return tells you, after the fact, how good the action you took was, and you weight your gradient accordingly. The loss surgery is the same; the labels and the per-sample weights come from different places. This is the conceptual bridge for a reader of `scratchnn`: the math you already know carries over; what changes is who provides the labels (you vs the environment) and how the per-sample importance is scored (uniformly vs by return).

7. Worked example: REINFORCE on a 5×5 gridworld

The full implementation is `examples/reinforce_gridworld.py`. The file is standalone NumPy, around 250 lines. The reason it is NumPy and not pure-Python `scratchnn` is the same honest acknowledgement the Transformer post makes: a policy-gradient run does 2,000 episodes of up to 25 steps each, so roughly 5×10^4 forward passes. The per-step Python overhead of the `scratchnn` core would dominate the actual learning. The MLP itself is the same MLP from posts 1 to 5, just vectorized. The interesting object is no longer the layer; it is the loss.

Why gridworld

A multi-armed bandit has no state and collapses credit assignment to one step. `CartPole` has continuous state and a dense reward, which hides the credit-assignment problem behind a forgiving reward signal. A 5×5 gridworld has a small discrete state space, a sparse reward (most of the reward arrives at the goal), and an obvious visualization. Credit assignment is on the page: you can see the value of progress propagate backward from the goal as training proceeds.

Setup

- **State.** Agent position $(r, c) \in \{0, \dots, 4\}^2$. One-hot-encoded as a 25-dim vector.
- **Actions.** Up, down, left, right. Off-grid moves are no-ops.
- **Reward.** -0.01 per step, $+1.0$ on entering the goal cell. Episode ends at the goal or after 25 steps.

- **Discount.** $\gamma = 0.99$.
- **Policy.** MLP with one tanh hidden layer of 64 units, then a 4-way softmax over actions. The same architecture you would build in scratchnn with `Linear(25, 64) -> Tanh -> Linear(64, 4)` and a `SoftmaxCrossEntropy` head.
- **Update.** REINFORCE with a moving-average return baseline: $b_{\text{new}} = 0.9b_{\text{old}} + 0.1R_{\text{episode}}$.
- **Optimizer.** Plain SGD on the policy parameters, learning rate 0.05.

The training loop

In pseudocode, for each episode:

1. Roll out under π_θ : at each step sample $a_t \sim \pi_\theta(\cdot | s_t)$, observe r_{t+1} and s_{t+1} , append to the episode buffer.
2. Compute returns-to-go G_t from the trailing rewards (a single backward pass through the reward sequence).
3. Compute per-step weights $w_t = G_t - b$.
4. Update the policy: descend on $-\sum_t \log \pi_\theta(a_t | s_t) \cdot w_t$. This is softmax cross-entropy summed over the episode, reweighted per step. The gradient is `(softmax(z) - onehot(a))` at each row, scaled by w_t , exactly as derived in §6.

The gradient code in `examples/reinforce_gridworld.py` is one screen: a forward pass over the entire episode's one-hot states, a softmax, a subtract-onehot-and-scale step, then the standard MLP backward through two linear layers and a tanh.

Results

The optimal trajectory has length 8 (Manhattan distance from start to goal), so the optimal return is $1.0 - 0.01 \cdot 7 = 0.93$ (seven step-penalties before the goal-reward step).

Before training, under the randomly-initialized policy:

```
mean return  -0.1258
mean length  24.20 steps
success rate 11.5%
```

The agent wanders. About one trial in nine stumbles into the goal inside the 25-step budget. The expected return is negative because most episodes pay the per-step cost and hit the step limit without a reward.

The smoothed learning curve (mean return per 100 episodes, 2,000 episodes total):

Episode	Mean return
100	+0.736
200	+0.902
300	+0.910
500	+0.918
1000	+0.922
1500	+0.927
2000	+0.928

A fast climb in the first ~200 episodes (the first chance trajectory that reaches the goal puts strong positive weight on the actions that got it there), then a slow refinement as the policy concentrates on the shortest paths. After 2,000 episodes:

```
mean return  +0.9278
mean length   8.21 steps
success rate 100.0%
```

100% of sampled trajectories reach the goal, and the average length is barely above the optimal 8. The trained greedy policy traces a Manhattan path:

(0,0) -> (1,0) -> (2,0) -> (2,1) -> (2,2) -> (2,3) -> (3,3) -> (3,4) -> (4,4)

Eight steps, return +0.93, exactly optimal. The policy has learned to walk to the goal.

What is in the learning curve

The curve has the shape every policy-gradient run on a sparse-reward task has. While the agent never reaches the goal, every episode collects only the -0.01 per-step penalty, and the policy’s parameters drift on essentially uninformative gradients. The first chance trajectory that reaches the goal produces a strongly positive return, the REINFORCE update reinforces every action it took, and the policy now has a non-trivial probability of repeating that trajectory. Subsequent successful episodes refine it. The curve’s fast initial rise is the moment “the algorithm finds an algorithm”; the long tail is parameter tuning.

This is the smallest visible version of a phenomenon that haunts larger RL. With sparser rewards, longer horizons, or more actions, the chance trajectory that gives the first informative return can take much longer to arrive. Designing the reward to give partial credit (reward shaping, §9), or biasing exploration toward promising states (intrinsic curiosity, UCB), or initializing the policy from a behavioral-cloning warmup, are all interventions aimed at shortening that wait.

8. AIXI: the theoretical north star

Solomonoff induction, introduced in the language-model post, is the optimal incomputable *predictor*: a Bayesian mixture over all computable environments, weighted by program length under a universal prior. It is what an idealized agent would do if it could enumerate every Turing machine, score each by how well it explains the observations seen so far, and weight its predictions by the prior times the likelihood. Practical language models (Bengio, RNN, Transformer) are computable approximations to that ideal.

AIXI (Hutter, 2000) is its action-taking sibling. Where Solomonoff predicts, AIXI *acts*. The defining expression: at each step t with interaction history $h_{<t}$, AIXI selects

$$a_t^* = \arg \max_{a_t} \sum_{o_t r_t} \xi(o_t r_t \mid h_{<t} a_t) \cdot V^*(h_{<t} a_t o_t r_t),$$

where ξ is the Solomonoff prior over environments (a mixture over all programs that could have generated the observed history), o_t is the next observation, r_t is the next reward, and V^* is the optimal value function under that prior, defined recursively in the same way.

Decoded: AIXI maintains a posterior over all possible environments consistent with what it has seen so far, and at every step takes the action whose *expected* future discounted reward, averaged

over that posterior, is highest. The expectation is taken over the infinite-horizon trajectory under the optimal policy in each candidate environment, which is itself an AIXI-style recursion.

AIXI is literally Solomonoff induction plus Bayesian decision theory plus reward maximization, fastened together. Each piece is a component the dedicated RL series will unpack on its own:

- The Solomonoff prior over environments: a universal Bayesian prior, computable to no finite degree, but the right ideal for a learner that wants to be robust to model misspecification.
- Bayesian decision theory: the action that maximizes posterior expected utility is the action a coherent agent should take.
- Reward maximization: the agent’s utility is a sum of discounted rewards. (This is itself a strong commitment. See §9.)

The parallel. The supervised arc had Solomonoff induction as the unachievable but illuminating prediction ideal; modern LMs are practical computable approximations. RL has AIXI as the unachievable but illuminating action ideal; modern RL algorithms (REINFORCE, DQN, PPO, MuZero, decision transformers) are practical computable approximations. In both arcs the architecture is the computable inductive bias substituted for an incomputable optimum. You cannot run Solomonoff. You can run a Transformer. You cannot run AIXI. You can run PPO. The architecture *is* where you compromise.

Reading AIXI as a target sharpens what each piece of practical RL is doing. Q-learning learns a *single* environment model implicitly, where AIXI averages over all of them. PPO uses a parameterized policy network and clipped surrogate objectives, where AIXI uses an exact arg-max over an infinite expectation. Decision transformers collapse policy and value into a sequence-prediction problem, where AIXI keeps them separate. Each compromise can be located in the distance between the algorithm and the AIXI specification.

9. Inductive bias, the RL axis

The supervised series argued that architecture (posts 3 to 6) and output head (post 2) are parallel axes of inductive bias. The same frame extends to RL with more design surface, each piece of which encodes assumptions about the task.

1. **Reward shaping.** The reward function is a prior about the value landscape. A sparse goal-reward (the gridworld setup above) is minimal-prior: the agent has to find a successful trajectory before it gets any informative gradient. Shaping rewards (a small positive reward for getting closer to the goal, for instance) injects information about which intermediate states are progress. The shaping can also bias the policy in the wrong direction if your hint is wrong, which has its own literature (reward hacking, Goodhart’s law). The reward is a *teacher*; the choice of teacher is a prior.
2. **Policy architecture.** Whether π_θ is an MLP, a CNN over pixels, an RNN over partial observations, or a Transformer over state-action histories is the *same* architectural choice the supervised series catalogued. The CNN’s locality, the RNN’s recurrence, the Transformer’s content-addressable lookup all apply, just in service of action selection rather than prediction. The gridworld example uses an MLP because the state is small and discrete and there is no spatial or temporal structure to exploit. Atari from pixels uses a CNN. POMDPs with hidden state use an RNN or Transformer.
3. **Exploration strategy.** Epsilon-greedy, entropy bonus, intrinsic curiosity, UCB, Thompson

sampling, posterior sampling: each is a prior about where useful information is. Epsilon-greedy assumes “uniform random is good enough sometimes.” Curiosity assumes “novel states are valuable.” Thompson sampling assumes a posterior over models and exploits its variance. Different priors, different exploration behavior, and a real source of difference between algorithms on sparse-reward tasks.

4. **Algorithm class.** On-policy vs off-policy, model-free vs model-based, value vs policy vs actor-critic: each commits to a different structural assumption about what is cheap, what is reliable, and what generalizes. Off-policy methods assume past trajectories from different policies still inform the current policy. Model-based methods assume a learned dynamics model will pay off the cost of fitting it. Value-based methods assume the Bellman recursion is more reliable than direct policy parameterization. The choice is a prior about the structure of the task and the learning dynamics.

All of these are inductive biases in exactly the same sense that “translation equivariance” is for a CNN. The pattern from posts 2 to 6 generalizes to RL, with more axes. A real RL system is a stack of priors at every level (reward, policy architecture, exploration, algorithm), and matching each to the task is a separate engineering problem.

10. Closing

This is the end of the seven-post arc. The thread was: supervised learning plus a parametric model plus a loss equals a paradigm with two main axes of inductive bias, architecture and output head. The seven posts laid out concrete instances along both axes, from a 1-layer logistic regression to a transformer with stacked content-addressable lookup.

Reinforcement learning is what the rest of machine learning looks like when the per-example label disappears. It deserves its own arc, and a dedicated series is forthcoming. AIXI is the theoretical north star of that series, the way Solomonoff induction was for the language-modeling post. Practical RL algorithms (REINFORCE, DQN, PPO, MuZero, decision transformers) will be the computable approximations, each one locatable as a specific compromise away from the AIXI optimum.

Modern systems blur the supervised-vs-RL boundary, and the blur is itself worth naming. **RLHF** (reinforcement learning from human feedback) fine-tunes a language model with a reward model that was itself trained by supervised learning on human preference judgments; the inner loop of the reward model is pure supervised learning, the outer loop is policy gradient. **AlphaZero** uses supervised distillation against MCTS rollouts: the targets come from a search procedure, not human labels, and the policy and value networks are trained by supervised cross-entropy and MSE against them. **Decision transformers** cast offline RL as supervised sequence modeling on (return, s, a) tuples: a transformer is trained by ordinary language-model loss on trajectories, with the desired return passed in as a conditioning token at inference. The boundary is conceptual, not categorical, and a serious treatment threads back and forth across it.

The math you derived in posts 1 to 6 is what every one of these systems runs on. Softmax cross-entropy is the gradient signal in behavioral cloning, in policy gradient, in PPO’s clipped surrogate, in the policy network of AlphaZero, in the transformer-loss view of offline RL. MSE is the gradient signal in value-function regression, in the value head of AlphaZero, in the world model of MuZero, and in the reward-model fitting step of RLHF. The supervised toolkit is the inner loop of every modern RL system. What RL adds is the outer loop, the credit-assignment machinery, and the

trajectory distribution over which the inner loop's losses are weighted.

The dedicated RL series picks up from here. AIXI sets the north star; REINFORCE in this post is its smallest computable shadow.